ENGINEERING NOTEBOOK

# Gauntlet

*A CI/CD quality gate, measured*

**Author** Nikhil Sinha

**Repository** github.com/Nikhil201716/Gauntlet-Quality-Gate

**Stack** Python · Flask · Behave (BDD) · AST mutation testing · Bandit (SAST) · DAST probes · GitHub Actions

**Edition** 1.0

# How to read this book

This is a software-quality notebook built around one small Flask app and a five-layer quality gate -- unit tests, BDD acceptance tests, mutation testing, SAST and DAST. It is written for someone who tests or ships software and wants to know, concretely, what each kind of check catches, what it misses, and why a green test suite is not a safe release.

Every listing is extracted from the repository at build time and every number is read from the reports a real run wrote, so nothing here can drift from the code. The headline is a measurement with its baseline and its false-positive rate both reported.

# Contents


## 1. A green test suite is not a safe release

1.1 The finding

1.2 The whole grid

1.3 How to read the rest

## 2. Notekeeper: the application under test

2.1 What it does

2.2 The app factory and its routes

2.3 The deterministic seed

## 3. The rest of the routes

3.1 Search and the note view

3.2 The admin export

## 4. Autoescaping, and the defect that bypasses it

4.1 A constant template

## 5. The authorization rules

5.1 Rules as pure functions

5.2 The seam that matters

## 6. Storage and the one query that matters

6.1 Search, done right

## 7. Password hashing

7.1 The clean version

## 8. The headers the app sets

8.1 Configured once, applied everywhere

## 9. Layer 1 - unit tests

9.1 What the unit suite asserts

## 10. Inside the unit suite

10.1 Storage tests catch the scoping bug

10.2 Hashing tests walk past the weak hash

**11. The unit and BDD runners**

11.1 pytest and Behave as gates

**12. Layer 2 - BDD acceptance tests**

12.1 The feature file

12.2 The steps

**13. Why a vulnerable app still passes its tests**

13.1 Features work; security does not

**14. Layer 3 - mutation testing: the idea**

14.1 Tests for the tests

**15. The mutation engine**

15.1 The mutators

**16. The other mutators**

16.1 Flipping and or, bumping constants

**17. Running the suite against a mutant**

17.1 The runner

**18. The mutation gate, end to end**

18.1 Scoring a suite

**19. The toothless suite (defect T1)**

19.1 A plausible weak suite

**20. Layer 4 - SAST with Bandit**

20.1 What Bandit catches

**21. SAST house rules**

21.1 A custom rule for Flask XSS

**22. The SAST scan, end to end**

22.1 Custom rules and the combined scan

**23. What SAST cannot see**

23.1 Absence is invisible

## 24. Layer 5 - DAST

24.1 The probes

## 25. A probe in detail

25.1 The SQL-injection probe

## 26. The remaining probes

26.1 Three more probes

## 27. SAST and DAST: the same vulnerability, two lenses

27.1 Complementary, not redundant

## 28. A worked example: S1 through all five layers

28.1 Tracing the SQL-injectable search

## 29. The defect catalog

29.1 The shape of a defect

29.2 The gap, declared up front

## 30. A vulnerability entry in full

30.1 Edits and a swap

## 31. The harness

31.1 Applying a defect

31.2 Running the layers

## 32. Assembling the report

32.1 The per-defect detail

32.2 The summary

## 33. The control, in code

33.1 N1: a benign change

## 34. The control: clean copy and benign change

34.1 Two guards against a rigged result

## 35. Reading the matrix by class

35.1 Per class

## 36. Why layers multiply

36.1 Independent detection

37. The IDOR that escaped

37.1 One route forgot to ask

38. Closing the gap

38.1 What would have caught it

39. The quality gate in CI

39.1 One command, one exit code

40. The gate's verdict

40.1 A readable pass/fail

41. The mutation ratchet

41.1 A number that only goes up

42. Reproducibility

42.1 Determinism by construction

43. Context: the OWASP classes these map to

43.1 Why these defects and not others

44. Where each layer belongs in a pipeline

44.1 Not everything runs on every commit

45. What this does not establish

45.1 Limits

46. Exercises

46.1 Problems

46.2 Answers

47. Recap

47.1 The five sentences

48. Runbook

48.1 Install and run

48.2 What you should see

49. Appendix - glossary

## 49.1 Glossary

CHAPTER 1

# A green test suite is not a safe release

> *Five gate layers, ten injected defects. The passing unit and acceptance suites catch two of them. The full gate catches nine. One escapes all five.*

## 1.1 The finding

Gauntlet is a quality gate you can run and measure. A small Flask application -- Notekeeper, where people register, log in, write notes, search them, and an admin exports them -- is wrapped in five independent checks: unit tests, BDD acceptance tests, mutation testing, static analysis (SAST) and dynamic probing (DAST). A harness injects ten known defects one at a time and records which layer catches which.

The clean application passes all five layers, and its authorization core scores a perfect 1.00 on the mutation gate. It is a genuinely good suite. And yet:

Table 1.1 - the headline (reports/summary.json)

| Gate | Catches (of 10) | What it sees |
|---|---|---|
| green test suite (unit + bdd) | 2 | functional bugs only |
| full five-layer gate | 9 | + weak tests, code vulns, runtime vulns |
| escaped every layer | 1 | a business-logic authorization flaw (IDOR) |
| false positives on a benign change | 0 | none |

The green suite catches 2 of 10 because the only defects it can see are the two that break a feature. It is blind to an injectable query, a stored XSS, a hard-coded secret, MD5 password hashing, a missing admin check, absent security headers, and -- most tellingly -- a unit test that passes while asserting nothing. Each of those is caught, but only by the layer built to see it.

## 1.2 The whole grid

Every row is a real run: the harness patches the source, runs the five layers against the patched copy, and marks a layer as having caught the defect if that layer fails. A dot means the layer passed (did not notice); a check means it failed (caught it).

Table 1.2 - defect x layer (reports/catch_matrix.json)

| id | unit | bdd | mut | sast | dast | caught by | defect |
|---|---|---|---|---|---|---|---|
| L1 | · | ✓ | · | · | · | bdd | search ignores the query |
| L2 | ✓ | · | · | · | ✓ | unit,dast | search drops owner scoping |
| T1 | · | · | ✓ | · | · | mutation | toothless authz unit tests |
| S1 | · | · | · | ✓ | ✓ | sast,dast | SQL-injectable search |
| S2 | · | · | · | ✓ | ✓ | sast,dast | stored XSS on the note page |
| S3 | · | · | · | ✓ | · | sast | hard-coded secret |
| S4 | · | · | · | ✓ | · | sast | MD5 password hashing |
| S5 | · | · | · | · | ✓ | dast | missing admin authorization |
| S6 | · | · | · | · | ✓ | dast | missing security headers |
| G1 | · | · | · | · | · | — none — | IDOR on the note route |
| N1 | · | · | · | · | · | — none — | benign display constant |

L1 and L2 are functional bugs; T1 is a weakened test; S1-S6 and G1 are vulnerabilities; N1 is a benign control. Of 10 defects, 9 are caught by at least one layer and 1 (G1) slips past all five. The benign change trips nothing.

> **EVERY CLAIM IS CHECKED**
>
> The catalog records, for each defect, which layers it predicts will catch it. The harness compares prediction to measurement: 10 of 10 agree (reports/agreement.json). The grid is the project holding itself to its own claims.

## 1.3 How to read the rest

The notebook tours the application, then each of the five layers in turn -- what it is, how it is implemented here, and exactly which defect class it owns -- then the harness that measures them, the defect that escaped, and the limits of the whole exercise.

- ['The app under test and its security-critical parts.', 'The five layers, one class of defect at a time, with the code of each.', 'The catalog, the harness, and reading the matrix.', 'The IDOR that escaped, and how to close it.', 'CI wiring, reproducibility, OWASP context, limits, exercises.']

CHAPTER 2

# Notekeeper: the application under test

> *An ordinary Flask app, because ordinary is where defects live.*

## 2.1 What it does

Notekeeper is deliberately unremarkable. A user registers and logs in; writes short notes; searches their own notes by title; and an administrator can export every note. That small surface is enough to host every class of defect the gate is meant to catch: a search query (SQL), a rendered note (XSS), a stored password (crypto), a config secret, an admin-only route (authorization) and response headers.

The clean application is the secure version of each of those. Every defect in the catalog is one surgical edit away from this code, which is the point -- these are not exotic bugs, they are one wrong line in an app that otherwise works and passes its tests.

## 2.2 The app factory and its routes

The app is built by a factory so every test gets a fresh, freshly seeded instance. The security-relevant decisions are marked in the source with the defect each one is the target of.

Listing 2.1 - the factory, headers hook and register route app/web.py

```
def create_app(seed: bool = True) -> Flask:
    app = Flask(__name__)
    app.secret_key = config.SECRET_KEY
    con = db.connect()
    app.config["DB"] = con
    if seed:
        _seed(con)

    def current_user() -> dict | None:
        uid = session.get("uid")
        if uid is None:
            return None
        row = db.get_user_by_id(con, uid)
        return dict(row) if row is not None else None

    @app.after_request
    def _apply_security_headers(response):
        for name, value in config.SECURITY_HEADERS.items():
            response.headers[name] = value
        return response

    @app.route("/register", methods=["GET", "POST"])
    def register():
        if request.method == "POST":
            username = request.form.get("username", "").strip()
            password = request.form.get("password", "")
            if not username or not password:
                return render_template_string(REGISTER_TMPL, error="both fields required")
            if db.get_user(con, username) is not None:
                return render_template_string(REGISTER_TMPL, error="username taken")
            uid = db.create_user(con, username, password)
            session["uid"] = uid
            return redirect(url_for("dashboard"))
```

A signed session cookie carries the logged-in user id; `current_user()` turns it back into a row. The `after_request` hook adds the security headers (defect S6 removes them). Note that creating a user here is the app's own flow against an in-memory database -- no real credentials are involved.

## 2.3 The deterministic seed

Two ordinary users and an admin, with notes, are seeded on every boot. The two ordinary users exist so a probe can prove a cross-user leak: if alice's request ever returns one of bob's notes, something is broken.

Listing 2.2 - the seed (app/web.py) app/web.py

```
def _seed(con) -> None:
    """Deterministic seed: an admin and two ordinary users with notes.

    The two ordinary users exist so a dynamic probe can prove cross-user data
    leakage (SQL injection, IDOR): if alice's request ever returns one of bob's
    notes, something is broken.
    """
    admin = db.create_user(con, "root", "root-pw", is_admin=1)
    alice = db.create_user(con, "alice", "alice-pw")
    bob = db.create_user(con, "bob", "bob-pw")
    db.add_note(con, admin, "ops runbook", "restart the worker pool nightly")
    db.add_note(con, alice, "groceries", "milk, eggs, bread")
    db.add_note(con, alice, "ideas", "a quality gate that measures itself")
    db.add_note(con, bob, "taxes", "file before April, keep the receipts")
    db.add_note(con, bob, "secret plan", "bob's confidential note, not for alice")
```

CHAPTER 3

# The rest of the routes

> *Search, the note view and the admin export -- the three security surfaces, in full.*

## 3.1 Search and the note view

The search route hands the query to `db.search_notes` and lists the results; the note view loads a note, checks `can_view`, and renders it through a constant, autoescaped template. Four defects target these few lines -- L1, S1, S2 and G1.

Listing 2.3 - the note view and search (app/web.py) app/web.py

```
@app.get("/note/<int:note_id>")
def view_note(note_id):
    user = current_user()
    if user is None:
        return redirect(url_for("login"))
    note = db.get_note(con, note_id)
    if note is None:
        abort(404)
    if not can_view(note, user):
        abort(403)
    return render_template_string(NOTE_TMPL, note=note, user=user)

@app.get("/search")
def search():
    user = current_user()
    if user is None:
        return redirect(url_for("login"))
    q = request.args.get("q", "")
    results = db.search_notes(con, user["id"], q)
    return render_template_string(SEARCH_TMPL, q=q, notes=results)
```

## 3.2 The admin export

The admin export is guarded by `is_admin`; defect S5 removes that guard, and only a runtime probe notices.

Listing 2.4 - the admin export (app/web.py) app/web.py

```
@app.get("/admin/export")
def admin_export():
    user = current_user()
    if not is_admin(user):
        abort(403)
    notes = db.all_notes(con)
    return render_template_string(ADMIN_TMPL, notes=notes)
```

CHAPTER 4

# Autoescaping, and the defect that bypasses it

> *Why the clean render is safe and S2 is not.*

## 4.1 A constant template

The note view renders a constant template string. Jinja autoescapes `{{ note['body'] }}`, so a body of `` is shown as text, not executed. The template is a literal, so the SAST rule leaves it alone.

Listing 2.5 - the autoescaped note template (app/web.py) app/web.py

```
NOTE_TMPL = _PAGE.format(body="""
<p><a href="{{ url_for('dashboard') }}">&larr; back</a></p>
<h3>{{ note['title'] }}</h3>
<div class=body>{{ note['body'] }}</div>""")
```

Defect S2 throws this away and builds the HTML by concatenating `note['body']` into a string passed to `render_template_string`. That is a dynamic template -- the body is now part of the template source, not an escaped value -- so the script runs. The custom SAST rule flags the concatenation; the DAST probe confirms the script survives. The clean template is the control that makes both findings meaningful.

CHAPTER 5

# The authorization rules

> *Small, pure functions -- the security the whole app rests on, and the substrate the mutation gate judges.*

## 5.1 Rules as pure functions

Who may see a note, who may edit it, who is an admin: these are the rules everything else depends on. Keeping them as small pure functions -- no framework, no database -- makes them easy to test exhaustively, which is why they are the natural target for mutation testing later.

Listing 3.1 - app/authz.py — app/authz.py

```
def is_admin(user: Mapping | None) -> bool:
    """True iff ``user`` is a signed-in administrator."""
    return user is not None and user["is_admin"] == 1


def owns(note: Mapping, user: Mapping | None) -> bool:
    """True iff ``user`` is the owner of ``note``."""
    return user is not None and note["owner_id"] == user["id"]


def can_view(note: Mapping, user: Mapping | None) -> bool:
    """A note may be viewed by its owner or by any admin."""
    return owns(note, user) or is_admin(user)


def can_edit(note: Mapping, user: Mapping | None) -> bool:
    """A note may be edited only by its owner -- admins may read, not rewrite."""
    return owns(note, user)


def visible_notes(notes: Sequence[Mapping], user: Mapping | None) -> list:
    """Filter ``notes`` down to the ones ``user`` is allowed to see."""
    return [n for n in notes if can_view(n, user)]
```

## 5.2 The seam that matters

Read `can_view` carefully: a note is visible to its owner or to an admin. `can_edit` is stricter -- only the owner, so an admin may read but not rewrite. These distinctions are exactly what a good test suite must pin down, and exactly what a weak one lets slide.

#### CORRECT AND TESTED IS NOT THE SAME AS WIRED IN

The whole capstone lesson lives in the gap between this module and the routes that call it. These rules can be perfect and fully tested, and the app can still leak -- if a single route forgets to call `can_view`. Hold that thought; it is the defect that escapes the entire gate.

CHAPTER 6

# Storage and the one query that matters

> *Parameterised and owner-scoped -- the safe, boring way, and the target of two different defects.*

## 6.1 Search, done right

Notes live in SQLite. Search is the one query where a mistake is dangerous, so the clean version does two things at once: it binds the search term as a value (never as SQL text) and it scopes the result to the caller.

Listing 4.1 - app/db.py search_notes — app/db.py

```
def search_notes(con: sqlite3.Connection, owner_id: int,
                 query: str) -> list[sqlite3.Row]:
    """Return the caller's own notes whose title matches ``query``.

    Parameterised and owner-scoped: the search term is a *bound value*, never
    part of the SQL text, so ``' OR '1'='1`` is matched as a literal title
    fragment and finds nothing.
    """
    sql = "SELECT * FROM notes WHERE owner_id = ? AND title LIKE ? ORDER BY id"
    return con.execute(sql, (owner_id, f"%{query}%")).fetchall()
```

Two defects target this one function. S1 rebuilds it as an f-string (SQL injection). L2 drops the `owner_id = ?` clause (a scoping bug that leaks across users without any injection). They are caught by different layers -- which is the first hint that one tool is not enough.

CHAPTER 7

# Password hashing

> *PBKDF2 with a salt -- and why a login test cannot tell it from MD5.*

## 7.1 The clean version

Passwords are stored as a salted PBKDF2-HMAC-SHA256 digest and checked in constant time. Defect S4 replaces both functions with a bare MD5.

Listing 5.1 - app/security.py — app/security.py

```
def hash_password(password: str, salt: bytes | None = None) -> str:
    """Return ``"<salt_hex>$<digest_hex>"`` for storage."""
    if salt is None:
        salt = os.urandom(16)
    digest = hashlib.pbkdf2_hmac(
        "sha256", password.encode("utf-8"), salt, PBKDF2_ITERATIONS
    )
    return salt.hex() + "$" + digest.hex()


def verify_password(password: str, stored: str) -> bool:
    """Constant-time check of ``password`` against a stored ``hash_password`` value."""
    salt_hex, _, digest_hex = stored.partition("$")
    try:
        salt = bytes.fromhex(salt_hex)
    except ValueError:
        return False
    check = hashlib.pbkdf2_hmac(
        "sha256", password.encode("utf-8"), salt, PBKDF2_ITERATIONS
    )
    return hmac.compare_digest(check.hex(), digest_hex)
```

**THE TEST THAT CANNOT SEE THE DIFFERENCE**

Login works identically whether the stored hash is PBKDF2 or MD5: the right password verifies, the wrong one does not. A round-trip unit test passes either way. That is why S4 is caught only by static analysis -- a tool that reads the code and recognises `hashlib.md5` -- and never by a functional test.

CHAPTER 8

# The headers the app sets

> *A safe default that a one-line defect silently removes.*

## 8.1 Configured once, applied everywhere

The security headers live in config and are applied to every response by the `after_request` hook shown earlier. They are the cheap, standard mitigations: stop MIME sniffing, forbid framing, constrain script sources, drop the referrer.

Listing 5.2 - the security headers (app/config.py) app/config.py

```
SECURITY_HEADERS = {
    "X-Content-Type-Options": "nosniff",
    "X-Frame-Options": "DENY",
    "Content-Security-Policy": "default-src 'self'",
    "Referrer-Policy": "no-referrer",
```

Defect S6 makes the hook return the response without adding any of them. Nothing in the source looks wrong -- the dict and the hook still exist -- so no static rule fires. Only fetching a real response and finding the headers absent reveals it, which is why S6 is a DAST-only catch.

CHAPTER 9

# Layer 1 - unit tests

> *A capable suite that pins the authorization rules branch by branch.*

## 9.1 What the unit suite asserts

The unit layer is pytest over the pure logic: the authorization rules, password hashing, and storage. The authorization tests are written to pin every branch -- owner, admin, other user, anonymous -- so the clean suite scores 100% on the mutation gate.

Listing 6.1 - the can_view tests (tests/test_authz.py) `tests/test_authz.py`

```
# --- can_view ----------------------------------------------------------------

def test_can_view_owner_sees_own_note():
    assert can_view(note(1), ALICE) is True


def test_can_view_admin_sees_other_users_note():
    assert can_view(note(2), ROOT) is True


def test_can_view_other_user_is_denied():
    assert can_view(note(2), ALICE) is False


def test_can_view_anonymous_is_denied():
    assert can_view(note(1), None) is False
```

Across the project the unit suite catches exactly 1 of 10 injected defects: L2, the search-scoping bug, because `test_db` asserts search is owner-scoped. It catches none of the security defects -- not because it is weak, but because they live in wiring and in properties these tests do not assert.

CHAPTER 10

# Inside the unit suite

> *The storage and hashing tests the security defects walk past.*

## 10.1 Storage tests catch the scoping bug

The storage tests assert the functional contract of search: it finds the caller's matching notes and is scoped to the caller. The second assertion is what catches defect L2.

Listing 6.3 - the scoping test (tests/test_db.py) tests/test_db.py

```
def test_search_is_scoped_to_owner():
    con, alice, bob = _fresh()
    rows = db.search_notes(con, alice, "grocer")
    assert all(r["owner_id"] == alice for r in rows)
    assert "groceries for bob" not in {r["title"] for r in rows}
```

## 10.2 Hashing tests walk past the weak hash

The hashing tests check a round trip: the right password verifies, the wrong one does not, a malformed stored value is rejected. All three pass for MD5 too -- which is why S4 is a SAST catch, not a unit catch.

Listing 6.4 - the hashing tests (tests/test_security.py) tests/test_security.py

```
def test_round_trip_accepts_correct_password():
    stored = hash_password("correct horse battery staple")
    assert verify_password("correct horse battery staple", stored) is True


def test_rejects_wrong_password():
    stored = hash_password("s3kret")
    assert verify_password("not it", stored) is False


def test_malformed_stored_value_is_rejected():
    assert verify_password("x", "not-a-valid-hash") is False
```

CHAPTER 11

# The unit and BDD runners

> *Turning two test tools into gate verdicts.*

## 11.1 pytest and Behave as gates

The unit gate runs pytest and reads its exit code; the BDD gate runs Behave with the JSON formatter and inspects each scenario's status, so a failure names the scenario that broke.

Listing 6.2 - the unit runner (gate/unit.py) gate/unit.py

```
def run(cwd: str | None = None) -> dict:
    # pytest.ini already supplies -q; adding another would become -qq and
    # suppress the "N passed" summary line we parse here.
    proc = subprocess.run(
        [sys.executable, "-m", "pytest", "-p", "no:cacheprovider"],
        capture_output=True, text=True, cwd=cwd,
    )
    out = proc.stdout + proc.stderr
    passed = int(m.group(1)) if (m := re.search(r"(\d+) passed", out)) else 0
    failed = int(m.group(1)) if (m := re.search(r"(\d+) failed", out)) else 0
    return {
        "passed": proc.returncode == 0 and failed == 0,
        "n_passed": passed,
        "n_failed": failed,
```

Listing 7.3 - the BDD runner (gate/bdd.py) gate/bdd.py

```
def run(cwd: str | None = None) -> dict:
    fd, path = tempfile.mkstemp(suffix=".json")
    os.close(fd)
    try:
        subprocess.run(
            [sys.executable, "-m", "behave", "-f", "json", "-o", path, "-q",
             "--no-summary"],
            capture_output=True, text=True, cwd=cwd,
        )
        with open(path, encoding="utf-8") as fh:
            data = json.load(fh)
    except (json.JSONDecodeError, FileNotFoundError):
        return {"passed": False, "n_scenarios": 0, "failed": ["<behave did not run>"]}
    finally:
        if os.path.exists(path):
            os.unlink(path)

    scenarios = []
    for feature in data:
        for element in feature.get("elements", []):
            if element.get("type") == "scenario":
                scenarios.append((element["name"], element.get("status")))
    failed = [name for name, status in scenarios if status != "passed"]
    return {"passed": not failed, "n_scenarios": len(scenarios), "failed": failed}
```

CHAPTER 12

# Layer 2 - BDD acceptance tests

> *Business-language scenarios in Gherkin, run by Behave.*

## 12.1 The feature file

BDD (behaviour-driven development) writes acceptance tests in Gherkin -- Given/When/Then in business language -- so they read as a specification. Behave runs them against the real app through its test client.

Listing 7.1 - features/notekeeper.feature — `features/notekeeper.feature`

```
Scenario: a note I write shows up on my dashboard
  Given I am logged in as "alice"
  When I create a note titled "shopping list" with body "milk and eggs"
  And I open my dashboard
  Then I see "shopping list"

Scenario: search returns matching notes and hides the rest
  Given I am logged in as "alice"
  When I search for "groc"
  Then I see "groceries"
  And I do not see "ideas"

Scenario: my dashboard shows only my own notes
  Given I am logged in as "alice"
  When I open my dashboard
  Then I see "groceries"
  And I do not see "taxes"
```

## 12.2 The steps

Each Given/When/Then maps to a step function. The steps drive the Flask test client -- a real HTTP round trip, in process.

Listing 7.2 - a step definition (features/steps/steps.py) — `features/steps/steps.py`

```
@given('I am logged in as "{username}"')
def step_login(context, username):
    r = context.client.post(
        "/login", data={"username": username, "password": _PASSWORDS[username]}
    )
    assert r.status_code in (200, 302), f"login failed: {r.status_code}"
```

The acceptance suite catches 1 of 10: L1, where search ignores the query, breaks the scenario that says a search must hide non-matching notes. That is precisely what acceptance tests are good at -- and precisely the limit explored next.

CHAPTER 13

# Why a vulnerable app still passes its tests

> *The uncomfortable overlap: green unit + green BDD, and an exploitable app.*

## 13.1 Features work; security does not

Walk the security defects against the two functional layers. The SQL-injectable search (S1) still returns your matching notes for a normal query, so every search scenario passes. The stored XSS (S2) still shows the note text, so the view scenario passes. MD5 hashing (S4) still logs you in. The hard-coded secret (S3) changes no behaviour at all. The missing admin check (S5) only makes the admin export *more* permissive, so the admin's own scenario still passes. Absent headers (S6) are invisible to a test that only reads the body.

That is why the green suite catches 2 of 10. A functional test asserts that a feature works. A vulnerability, almost by definition, does not stop the feature from working -- it adds a capability the attacker wanted and the user never sees. The remaining three layers each exist to see a kind of wrong that 'the feature works' cannot.

#### THE BASELINE IS A CONTROL, NOT A STRAW MAN

It would be easy to make this point with a bad test suite. This one is good: 23 unit tests, four acceptance scenarios, and a 100% mutation score on the authorization core. The point stands anyway -- a strong green suite still misses whole classes of defect.

CHAPTER 14

# Layer 3 - mutation testing: the idea

> *Measure whether the tests would notice if the code were wrong.*

## 14.1 Tests for the tests

A green suite tells you the tests pass. It does not tell you they would fail if the code were broken. Mutation testing answers that directly: change the code under test in a small, deliberate way (a 'mutant') and rerun the suite. If a test now fails, the mutant is *killed* -- the suite had teeth there. If every test still passes, the mutant *survived*, and a real bug of that shape would have slipped through too.

$$MS = \frac{K}{M}$$

The mutation score MS is the fraction of M mutants that the suite killed (K). The gate passes when the score clears a threshold:

$$\text{pass} \Leftrightarrow MS \geq \tau$$

Here the threshold is 0.90, set just below the clean suite's 1.00 so that any genuine loss of coverage fails the gate. The engine mutates the authorization rules -- small, pure and security-critical, the right place to demand real teeth.

CHAPTER 15

# The mutation engine

> *A standard-library AST transformer: one deliberate change per mutant.*

## 15.1 The mutators

The engine parses the module to an abstract syntax tree and walks it in a fixed order, producing one mutant per mutation point: swap a comparison operator, flip a boolean operator, bump an integer constant, invert a boolean constant.

Listing 10.1 - a mutator (gate/mutation.py) gate/mutation.py

```
def visit_Compare(self, node: ast.Compare):
    self.generic_visit(node)
    if len(node.ops) == 1 and type(node.ops[0]) in _CMP_SWAP:
        if self._hit():
            old = type(node.ops[0]).__name__
            node.ops[0] = _CMP_SWAP[type(node.ops[0])]()
            self.label = f"cmp {old}->{type(node.ops[0]).__name__} @L{node.lineno}"
```

One change at a time, deterministic order, so the mutant set -- and therefore the score -- is reproducible.

Listing 10.2 - enumerating mutants gate/mutation.py

```
def generate_mutants(source: str) -> list[tuple[str, str]]:
    """Return ``[(label, mutated_source), ...]`` -- one mutant per mutation point."""
    mutants = []
    for k in range(_count_points(source)):
        m = _Mutator(k)
        tree = m.visit(ast.parse(source))
        ast.fix_missing_locations(tree)
        mutants.append((m.label, ast.unparse(tree)))
    return mutants
```

CHAPTER 16

# The other mutators

> *Boolean operators and constants.*

## 16.1 Flipping and or, bumping constants

Besides comparisons, the engine flips boolean operators and alters constants. The `and` in `user is not None and user['is_admin'] == 1` becomes `or`; the `1` becomes `2`. Each is a plausible typo, and a thorough suite kills all of them.

Listing 10.3 - the boolean-operator mutator (gate/mutation.py) gate/mutation.py

```
def visit_BoolOp(self, node: ast.BoolOp):
    self.generic_visit(node)
    if self._hit():
        old = type(node.op).__name__
        node.op = ast.Or() if isinstance(node.op, ast.And) else ast.And()
        self.label = f"bool {old}->{type(node.op).__name__} @L{node.lineno}"
```

It is the `and`/`or` mutants that defect T1's weak suite lets survive: they only misbehave when the user is anonymous or an admin, and the weak suite never passes either input.

CHAPTER 17

# Running the suite against a mutant

> *In-process, fast, isolated.*

## 17.1 The runner

For each mutant the engine loads the mutated module as `app.authz`, then executes the test file against it and counts failures. It runs in process -- no subprocess per mutant -- so the whole score is a fraction of a second.

Listing 11.1 - running tests against a mutant — gate/mutation.py

```
def _run_suite(authz_source: str, test_source: str) -> tuple[int, int]:
    """Run the test functions against ``authz_source`` in-process.

    Returns ``(n_tests, n_failures)``. A test "fails" if it raises anything.
    """
    app_pkg = types.ModuleType("app")
    app_pkg.__path__ = []  # mark as a package
    authz_mod = types.ModuleType("app.authz")
    saved = {k: sys.modules.get(k) for k in ("app", "app.authz")}
    sys.modules["app"] = app_pkg
    sys.modules["app.authz"] = authz_mod
    try:
        exec(compile(authz_source, "app/authz.py", "exec"), authz_mod.__dict__)
        ns: dict = {}
        exec(compile(test_source, "tests/test_authz.py", "exec"), ns)
        tests = [v for k, v in sorted(ns.items())
                 if k.startswith("test_") and callable(v)]
        failures = 0
        for t in tests:
            try:
                t()
            except Exception:
                failures += 1
        return len(tests), failures
```

The score is computed only if the unmutated baseline is green -- mutation testing of a red suite is meaningless. On the clean app the engine generates its mutants and the suite kills all of them: MS = 1.00.

CHAPTER 18

# The mutation gate, end to end

> *From a module path to a pass/fail verdict.*

## 18.1 Scoring a suite

`mutation_score` ties the engine together: confirm the baseline is green, generate the mutants, run the suite against each, and return the score with the surviving mutants named so a failure is actionable.

Listing 11.2 - mutation_score (gate/mutation.py) gate/mutation.py

```
def mutation_score(authz_path: str, tests_path: str) -> dict:
    authz_source = Path(authz_path).read_text(encoding="utf-8")
    test_source = Path(tests_path).read_text(encoding="utf-8")

    n_tests, base_fail = _run_suite(authz_source, test_source)
    if base_fail:
        return {"baseline_green": False, "baseline_failures": base_fail,
                "n_tests": n_tests, "total": 0, "killed": 0,
                "kill_rate": 0.0, "survivors": [], "passed": False}

    mutants = generate_mutants(authz_source)
    killed, survivors = 0, []
    for label, msrc in mutants:
        _, fails = _run_suite(msrc, test_source)
        if fails:
            killed += 1
        else:
            survivors.append(label)
    total = len(mutants)
    rate = killed / total if total else 1.0
    return {
        "baseline_green": True,
        "n_tests": n_tests,
        "total": total,
        "killed": killed,
        "kill_rate": round(rate, 4),
        "survivors": survivors,
        "threshold": MUTATION_THRESHOLD,
        "passed": rate >= MUTATION_THRESHOLD,
```

The survivors list is the useful part on a failure: it names exactly which deliberate bug the suite failed to catch, which points straight at the missing assertion.

CHAPTER 19

# The toothless suite (defect T1)

> *Still green, but the score falls to 0.75 and the gate fails.*

## 19.1 A plausible weak suite

Defect T1 swaps the thorough authorization tests for a shallow set -- the two cases a developer thinks of first, and no more. It never passes `None` (anonymous) and never checks the admin override.

Listing 12.1 - the weak suite (harness/variants/) harness/variants/test_authz_weak.py

```
def test_owner_can_view_own_note():
    assert can_view(note(1), {"id": 1, "is_admin": 0})


def test_other_user_cannot_view():
    assert not can_view(note(2), {"id": 1, "is_admin": 0})


def test_owns_is_true_for_owner():
    assert owns(note(1), {"id": 1, "is_admin": 0})
```

This suite passes -- pytest is green. But the mutation score falls from 1.00 to 0.75: the boolean-operator mutants that only misbehave on an anonymous or admin input now survive, because no test exercises those inputs. 0.75 is below the 0.90 threshold, so the gate fails.

#### GREEN IS NOT TESTED

T1 is caught by exactly one layer: mutation. The unit and BDD suites are both green on it, because a weakened test is still a passing test. Only a tool that measures whether the tests have teeth can tell a thorough suite from a decorative one.

CHAPTER 20

# Layer 4 - SAST with Bandit

> *Read the code; flag the dangerous shapes.*

## 20.1 What Bandit catches

Static application security testing reads source without running it. Bandit is the standard Python security linter; the gate runs it over `app/` and treats any finding as a failure, because the clean code produces none.

Table 13.1 - the three Bandit catches

| Defect | Bandit rule | What it saw |
|---|---|---|
| S1 SQL injection | B608 | SQL statement built by string formatting |
| S3 hard-coded secret | B105 | a secret assigned as a literal |
| S4 MD5 hashing | B324 | a weak hash used for security |

Listing 13.1 - invoking Bandit (gate/sast.py) gate/sast.py

```
def run_bandit(app_dir: str) -> list[dict]:
    """Return Bandit findings as ``[{id, severity, confidence, file, line, text}]``."""
    proc = subprocess.run(
        [sys.executable, "-m", "bandit", "-r", app_dir, "-f", "json", "-q"],
        capture_output=True, text=True,
    )
    try:
        data = json.loads(proc.stdout or "{}")
    except json.JSONDecodeError:
        return []
    out = []
    for r in data.get("results", []):
        out.append({
            "tool": "bandit",
            "id": r["test_id"],
            "severity": r["issue_severity"],
            "confidence": r["issue_confidence"],
            "file": os.path.basename(r["filename"]),
            "line": r["line_number"],
            "text": r["issue_text"].split("\n")[0][:90],
        })
    return out
```

### A VERSION GOTCHA WORTH RECORDING

Bandit before 1.9 crashes on Python 3.13/3.14: it reads the removed `node.s` attribute on string constants and throws on every file -- while reporting 'no issues', which looks like a clean pass. The fix is `bandit>=1.9`. A tool that silently scans nothing is worse than no tool.

CHAPTER 21

# SAST house rules

> *What Bandit misses, a few AST rules of your own can catch.*

## 21.1 A custom rule for Flask XSS

Bandit does not reliably flag the Flask XSS pattern, so the gate adds house rules -- the kind of project-specific check a team writes in semgrep. The key one flags a dynamic template passed to `render_template_string`, which bypasses Jinja autoescaping.

Listing 14.1 - the custom XSS rule (gate/sast.py) gate/sast.py

```
def visit_Call(self, node: ast.Call):
    fn = node.func
    name = fn.attr if isinstance(fn, ast.Attribute) else (
        fn.id if isinstance(fn, ast.Name) else "")
    if name == "render_template_string" and node.args and _is_dynamic(node.args[0]):
        self._add("XSS001", node.lineno,
                  "dynamic template passed to render_template_string (XSS/SSTI)")
    if name == "Markup" and node.args and _is_dynamic(node.args[0]):
        self._add("XSS002", node.lineno, "Markup() wrapping a built-up string")
    if name in ("eval", "exec"):
        self._add("CODE001", node.lineno, f"use of {name}()")
    self.generic_visit(node)
```

The clean note view passes a constant template, so it is not flagged. Defect S2 passes a string built by concatenation -- a `BinOp` -- which the rule catches (XSS001). With Bandit plus these rules, SAST catches 4 of 10: S1, S2, S3 and S4 -- the code-level vulnerabilities.

CHAPTER 22

# The SAST scan, end to end

> *Bandit plus house rules, combined into one verdict.*

## 22.1 Custom rules and the combined scan

The custom layer walks each file for dangerous calls and does a line-level pass for template flags like `|safe` and `autoescape=False`. `scan` unions the Bandit findings with the custom ones and fails on any.

Listing 14.2 - custom_findings and scan (gate/sast.py) gate/sast.py

```
def custom_findings(app_dir: str) -> list[dict]:
    findings: list[dict] = []
    for path in sorted(Path(app_dir).rglob("*.py")):
        source = path.read_text(encoding="utf-8")
        v = _RuleVisitor(path.name)
        v.visit(ast.parse(source))
        findings.extend(v.findings)
        # text-level checks for template flags that are not expressions
        for i, line in enumerate(source.splitlines(), 1):
            stripped = line.replace(" ", "")
            if "|safe" in stripped:
                findings.append({"tool": "custom", "id": "XSS003", "severity": "HIGH",
                                 "confidence": "MEDIUM", "file": path.name, "line": i,
                                 "text": "Jinja |safe filter disables autoescaping"})
            if "autoescape=False" in stripped:
                findings.append({"tool": "custom", "id": "XSS004", "severity": "HIGH",
                                 "confidence": "HIGH", "file": path.name, "line": i,
                                 "text": "autoescape disabled"})
    return findings


def scan(app_dir: str = "app") -> dict:
    findings = run_bandit(app_dir) + custom_findings(app_dir)
    findings.sort(key=lambda f: (f["file"], f["line"]))
    return {"findings": findings, "n": len(findings), "passed": len(findings) == 0}
```

CHAPTER 23

# What SAST cannot see

> *Two real vulnerabilities leave no dangerous pattern in the source.*

## 23.1 Absence is invisible

SAST matches patterns in code. But two of the defects are not a dangerous pattern -- they are a *missing* safe one. S5 removes the `is_admin` check from the admin export; the remaining code is ordinary and correct-looking. S6 stops the `after_request` hook from adding headers; again, nothing dangerous is written, something safe is simply gone.

No static rule flags S5 or S6 without guessing intent, which is how static analysis earns false positives. The honest answer is that these are runtime properties: you cannot see them by reading the code, only by running it and looking at what the app actually does. That is the next layer.

CHAPTER 24

# Layer 5 - DAST

> *Attack the running app and report what actually works.*

## 24.1 The probes

Dynamic application security testing drives the running app with malicious input and inspects the responses. The gate runs five probes through Flask's in-process test client -- no network, no server -- each returning a finding or nothing.

- ["**SQL injection** - search with an injection payload; flag if the results include another user's note.", "**Owner-scope** - search for a term only another user's note has; flag any leak (catches scoping bugs that are not injectable).", '**Reflected XSS** - store a scripted note, fetch it, flag if the script survives unescaped.', '**Broken access control** - call the admin export as a non-admin; flag a 200.', '**Security headers** - fetch a page and flag any missing header.']

DAST catches 5 of 10: S1 and S2 (confirming SAST), plus L2, S5 and S6 -- the runtime-only findings.

CHAPTER 25

# A probe in detail

> *Why the payload has to break out of the LIKE wrapper.*

## 25.1 The SQL-injection probe

Listing 17.1 - the SQLi probe (gate/dast.py) gate/dast.py

```
def probe_sql_injection(app) -> dict | None:
    """Alice searches with an injection payload; flag if she sees another owner's note."""
    c = app.test_client()
    _login(c, "alice", "alice-pw")
    alice_id = _uid(app, "alice")
    r = c.get("/search", query_string={"q": SQLI_PAYLOAD})
    owners = {int(o) for o in re.findall(r"owner (\d+)", r.get_data(as_text=True))}
    leaked = owners - {alice_id}
    if leaked:
        return {"probe": "sqli", "severity": "HIGH",
                "detail": f"search leaked notes owned by {sorted(leaked)} to alice"}
```

The payload is not the textbook `' OR '1'='1`. The query wraps the term in `'%...%'` for a LIKE, so a naive payload leaves a dangling `%'` that turns the injected comparison false. The working payload closes the quote and comments out the trailing wrapper, so the OR is always true and every row returns. Getting this wrong is why the first run of this project showed S1 caught by SAST but not DAST -- a reminder that a probe that does not fire proves nothing.

> **THE CLEAN APP IS THE CONTROL**
>
> The same payload, sent to the clean app, is bound as a literal LIKE value and matches no title, so the probe returns nothing. A probe is only evidence if it is silent on safe code and loud on unsafe code.

CHAPTER 26

# The remaining probes

> *Owner-scope, access control and headers.*

## 26.1 Three more probes

Beyond injection and XSS, three probes cover the runtime-only classes: an owner-scope leak, a broken access-control check, and missing headers.

Listing 16.1 - access-control and header probes (gate/dast.py) gate/dast.py

```
def probe_broken_access_control(app) -> dict | None:
    """Non-admin alice hits the admin export; flag any success."""
    c = app.test_client()
    _login(c, "alice", "alice-pw")
    r = c.get("/admin/export")
    if r.status_code == 200:
        return {"probe": "authz", "severity": "HIGH",
                "detail": "/admin/export returned 200 to a non-admin user"}
    return None


def probe_security_headers(app) -> dict | None:
    c = app.test_client()
    _login(c, "alice", "alice-pw")
    r = c.get("/")
    missing = [h for h in REQUIRED_HEADERS if h not in r.headers]
    if missing:
        return {"probe": "headers", "severity": "MEDIUM",
                "detail": f"missing security headers: {missing}"}
```

Each builds a fresh, freshly seeded app, so the probes cannot interfere with one another and the order does not matter.

CHAPTER 27

# SAST and DAST: the same vulnerability, two lenses

> *Where they overlap, where only one can see.*

## 27.1 Complementary, not redundant

Table 18.1 - the six vulnerabilities across the two gates

| Defect | SAST | DAST | why |
|---|---|---|---|
| S1 SQL injection | ✓ | ✓ | dangerous pattern AND exploitable |
| S2 stored XSS | ✓ | ✓ | dangerous pattern AND exploitable |
| S3 hard-coded secret | ✓ | · | visible in code, no runtime effect |
| S4 MD5 hashing | ✓ | · | visible in code, login still works |
| S5 missing admin check | · | ✓ | an absence; only runtime shows it |
| S6 missing headers | · | ✓ | an absence; only the response shows it |

S1 and S2 are caught twice -- genuine defense in depth: a static rule and a live exploit confirming each other. S3 and S4 are SAST-only, because they change no observable behaviour. S5 and S6 are DAST-only, because they are the absence of a safe thing rather than the presence of a dangerous one. Neither tool dominates; you need both.

CHAPTER 28

# A worked example: S1 through all five layers

> *One defect, five verdicts, traced by hand.*

## 28.1 Tracing the SQL-injectable search

Take S1 -- the search query rebuilt as an f-string -- and walk it past each layer, which is the clearest way to see why the matrix reads as it does.

1. ['**unit (pass).** `test_db` searches for a benign term as one user. The f-string still scopes to that user for a normal term, so the results are correct and the test passes. The vulnerability is invisible to a benign input.', "**bdd (pass).** Same story at the acceptance level: searching for 'groc' returns the grocery note and hides the rest. The feature works, so the scenario is green.", '**mutation (pass).** S1 edits `db.py`, not the authorization module or its tests, so the mutation score is unchanged at 1.00.', '**sast (FAIL).** Bandit reads the f-string with SQL keywords and raises B608 -- a SQL statement built by string formatting. No execution required.', "**dast (FAIL).** The probe searches with a payload that closes the LIKE quote and comments out the wrapper; the query returns every row, including another user's note, and the probe sees the leak."]

Two green, one unaffected, two red -- and the two reds are a static reader and a live attacker independently reaching the same verdict. That is the whole argument for a layered gate, in one defect.

CHAPTER 29

# The defect catalog

> *One known problem per entry, injected as a source patch, with a prediction the run must honour.*

## 29.1 The shape of a defect

Each entry is one or more edits to the clean source -- a literal find/replace, or a whole-file swap for the weak-test case. The `expected` field names the layers the catalog predicts will catch it; the harness measures what actually happens and checks the prediction.

Listing 19.1 - the Defect record (harness/defects.py) harness/defects.py

```
class Defect:
    id: str
    name: str
    klass: str
    is_defect: bool
    expected: tuple[str, ...]
    desc: str
    edits: tuple = field(default_factory=tuple)
```

The catalog spans four classes: `logic` (a feature is wrong), `weak-test` (the tests pass but assert nothing), `vuln` (a security hole) and `benign` (a harmless change, the control).

## 29.2 The gap, declared up front

The IDOR defect declares an empty prediction -- the catalog states plainly that no layer here catches it. The run confirms it.

Listing 19.2 - the IDOR entry predicts its own escape harness/defects.py

```
id="G1", name="IDOR on the note route", klass="vuln", is_defect=True,
expected=(),  # the honest gap: no layer here catches it
desc="the note view stops calling can_view, so any logged-in user can read "
     "any note by id. The authz rules are still correct and fully tested "
     "(mutation stays 100%); the app leaks anyway because one route forgot "
     "to call them -- and no layer in this gate happens to probe for it.",
edits=(Edit("app/web.py", _CLEAN_VIEW_RENDER,
            "        return render_template_string"
            "(NOTE_TMPL, note=note, user=user)"),),
```

CHAPTER 30

# A vulnerability entry in full

> *What one catalog edit looks like.*

## 30.1 Edits and a swap

A defect is a tuple of edits. An `Edit` is a literal find/replace in one file; a `FileSwap` replaces a whole file (used for the weak-test case, which swaps in a shallow suite).

Listing 19.3 - Edit and FileSwap (harness/defects.py) harness/defects.py

```
class Edit:
    """A literal find/replace in one file (relative to the repo root)."""
    file: str
    old: str
    new: str


@dataclass(frozen=True)
class FileSwap:
    """Replace ``file`` wholesale with the contents of ``source``."""
    file: str
    source: str
```

The SQL-injection entry is one edit that turns the parameterised query into an f-string -- and names the two layers it expects to catch it.

Listing 19.4 - the S1 entry (harness/defects.py) harness/defects.py

```
id="S1", name="SQL-injectable search", klass="vuln", is_defect=True,
expected=("sast", "dast"),
desc="search is rebuilt as an f-string, so owner_id and the term are "
     "interpolated into the SQL -- Bandit (B608) sees it statically and a "
     "probe confirms it leaks other users' notes.",
edits=(Edit("app/db.py", _CLEAN_SEARCH_DB,
            '    sql = f"SELECT * FROM notes WHERE owner_id = {owner_id} '
            "AND title LIKE '%{query}%' ORDER BY id\"\n"
            '    return con.execute(sql).fetchall()'),),
```

CHAPTER 31

# The harness

> *Copy, patch, run all five layers in isolation, record the catches.*

## 31.1 Applying a defect

For each defect the harness copies the whole repository to a temporary directory and applies the edits. A find/replace that does not match exactly once is a hard error -- so a catalog that drifts from the code fails loudly instead of silently testing nothing.

Listing 20.1 - applying a patch (harness/score.py) harness/score.py

```
def _apply(root: Path, defect: Defect) -> None:
    for edit in defect.edits:
        target = root / edit.file
        if isinstance(edit, FileSwap):
            source = root / edit.source
            target.write_text(source.read_text(encoding="utf-8"), encoding="utf-8")
        elif isinstance(edit, Edit):
            text = target.read_text(encoding="utf-8")
            count = text.count(edit.old)
            if count != 1:
                raise RuntimeError(
                    f"{defect.id}: edit target found {count} times in {edit.file} "
                    f"(expected exactly 1). The source drifted from the catalog."
                )
            target.write_text(text.replace(edit.old, edit.new), encoding="utf-8")
```

## 31.2 Running the layers

Each layer runs as its own subprocess with the patched copy as its working directory, so there is no import caching between runs and every layer sees the patched code. A layer caught the defect if it fails on the copy.

Listing 20.2 - the five layers per copy harness/score.py

```
# mutation / sast / dast: each prints a JSON verdict
out["mutation"] = _json_stdout(_py(["gate/mutation.py"], root))
out["sast"] = _json_stdout(_py(["gate/sast.py", "app"], root))
out["dast"] = _json_stdout(_py(["gate/dast.py"], root))
return out
```

CHAPTER 32

# Assembling the report

> *From per-defect rows to the headline summary.*

## 32.1 The per-defect detail

Each row of the matrix carries the per-layer pass/fail and, for the mutation layer, the score on that copy -- so the one row where the score drops is visible at a glance.

Table 20.1 - per-defect detail (reports/catch_matrix.json)

| id | class | mutation score | caught by |
|---|---|---|---|
| L1 | logic | 1.00 | bdd |
| L2 | logic | 1.00 | unit,dast |
| T1 | weak-test | 0.75 | mutation |
| S1 | vuln | 1.00 | sast,dast |
| S2 | vuln | 1.00 | sast,dast |
| S3 | vuln | 1.00 | sast |
| S4 | vuln | 1.00 | sast |
| S5 | vuln | 1.00 | dast |
| S6 | vuln | 1.00 | dast |
| G1 | vuln | 1.00 | — none — |
| N1 | benign | 1.00 | — none — |

Only T1 moves the mutation score -- from 1.00 everywhere else to 0.75. The security and logic defects do not touch the authorization module or its tests, so the score is unchanged; that is correct, not a bug, and it is why mutation is a narrow but sharp instrument.

## 32.2 The summary

`_assemble` reduces the rows to the headline: per-layer catch counts, the baseline-versus-gate comparison, what escaped, and the false-positive check.

Listing 20.3 - assembling the summary (harness/score.py) harness/score.py

```
per_layer = {layer: sum(1 for r in defects if layer in r["caught_by"])
             for layer in LAYERS}
baseline_caught = [r["id"] for r in defects
                   if any(l in r["caught_by"] for l in BASELINE_LAYERS)]
gate_caught = [r["id"] for r in defects if r["caught_by"]]
escaped = [r["id"] for r in defects if not r["caught_by"]]
false_positives = [r["id"] for r in benign if r["caught_by"]]

# agreement with the catalog's predictions
agree = []
for r in defects:
    agree.append({
        "id": r["id"],
        "expected": sorted(r["expected"]),
        "measured": sorted(r["caught_by"]),
        "match": sorted(r["expected"]) == sorted(r["caught_by"]),
    })

summary = {
    "n_defects": len(defects),
    "clean_passes_all_layers": not clean_caught,
    "clean_mutation_score": clean_layers["mutation"].get("kill_rate"),
    "per_layer_catches": per_layer,
    "baseline_green_suite": {
        "layers": list(BASELINE_LAYERS),
        "caught": baseline_caught,
        "n_caught": len(baseline_caught),
        "n_missed": len(defects) - len(baseline_caught),
    },
    "full_gate": {
        "caught": gate_caught,
        "n_caught": len(gate_caught),
```

CHAPTER 33

# The control, in code

> *A real edit that is not a defect.*

## 33.1 N1: a benign change

A fault-injection study needs a negative control: a change that edits the source but is not a defect, to prove the gate does not fire on anything that moves. N1 adds an unused display-name constant next to the config token.

Listing 21.1 - the benign control (harness/defects.py) harness/defects.py

```
id="N1", name="benign display constant", klass="benign", is_defect=False,
expected=(),
desc="a harmless, unused display-name constant -- the control: a change "
     "that is not a defect, to check the gate does not cry wolf.",
edits=(Edit("app/config.py", _CLEAN_TOKEN,
            _CLEAN_TOKEN + '\nAPP_NAME = "Notekeeper"  '
            "# benign display name, nothing security-relevant"),),
```

It is injected and scored exactly like a defect, and it trips nothing: 0 false positives across all five layers. Without this control, a reader could object that the gate simply flags any change; with it, the nine catches are credibly about the defects, not about the act of editing.

CHAPTER 34

# The control: clean copy and benign change

> *Why the run is not just confirmation bias.*

## 34.1 Two guards against a rigged result

A fault-injection study is only honest if it also measures its false positives and proves its baseline is capable. Gauntlet does both.

1. ['**The clean copy runs first.** Before any defect, the harness runs all five layers on the unpatched repo. The result records `clean_passes_all_layers = true`. If the clean app failed any layer, the whole matrix would be suspect.', '**A benign change (N1) is injected like any defect.** It adds an unused display-name constant -- a real edit that is not a defect. It must trip nothing, and it does: 0 false positives. A gate that cried wolf here would train its team to ignore it.']

These two controls are why the headline -- green suite 2, full gate 9, one escaped -- is a measurement rather than a slogan.

CHAPTER 35

# Reading the matrix by class

> *What each row teaches.*

## 35.1 Per class

**Logic bugs (L1, L2).** These are what the functional suites exist for. L1 breaks an acceptance scenario (BDD); L2 breaks a storage unit test and also leaks at runtime (unit + DAST). The green suite earns its keep here -- and only here.

**Weak test (T1).** Invisible to every layer except mutation, because it is not a bug in the app at all -- it is a bug in the confidence the tests give you.

**Code vulnerabilities (S1-S4).** SAST's territory. Two of them (S1, S2) are also exploitable at runtime, so DAST confirms them. Two (S3, S4) leave no runtime trace and are SAST-only.

**Runtime vulnerabilities (S5, S6).** DAST's territory. An absent check and absent headers -- no code pattern to match, only behaviour to observe.

**The escape (G1).** Its own chapter follows.

Net: the trivial 'is the suite green' gate catches 2 of 10; adding mutation, SAST and DAST raises it to 9. The marginal value of each added layer is exactly the class the cheaper layers could not see.

CHAPTER 36

# Why layers multiply

> *A back-of-envelope model for defense in depth.*

## 36.1 Independent detection

Think of each layer as having some probability of detecting a given defect class. If a defect reaches production only when every layer misses it, and the layers are roughly independent, the escape probability is the product of the miss probabilities:

$$P_{escape} = \prod_i (1 - d_i)$$

where d_i is layer i's detection probability for that defect class. The lesson of the matrix is that the d_i are nearly disjoint across classes: for a missing-header defect, the functional and SAST layers have d near zero while DAST has d near one. Adding a layer whose strength is another layer's blind spot drops the product sharply; adding a layer that overlaps an existing one barely moves it.

It also shows the limit of the model. For the IDOR class, every d_i in this gate is zero, so the product is one -- the defect escapes with certainty no matter how many of these five layers you stack. Independence does not help when everyone shares the same blind spot.

CHAPTER 37

# The IDOR that escaped

> *Correct, tested authorization rules -- and the app leaks anyway.*

## 37.1 One route forgot to ask

Defect G1 removes a single guard from the note route: the call to `can_view`. Any logged-in user can now read any note by id -- an insecure direct object reference (IDOR).

Listing 24.1 - the whole of defect G1

```
note = db.get_note(con, note_id)
        if note is None:
            abort(404)
-       if not can_view(note, user):
-           abort(403)
        return render_template_string(NOTE_TMPL, note=note, user=user)
```

Now trace it through the gate. The authorization rules in `authz.py` are untouched -- still correct, still scoring 100% on mutation. The unit and BDD suites are green: no scenario fetches another user's note by id. SAST sees nothing dangerous -- a removed function call is an absence, not a pattern. DAST's probes cover injection, XSS, the admin route and headers, but not this object-ownership check. So G1 is caught by nobody.

#### THE GAP, STATED PLAINLY

A perfect, fully tested authorization module does not make an app safe if a route forgets to call it. Unit tests of the helpers, a green acceptance suite, a 100% mutation score, a clean SAST report and a passing DAST run -- all five green, and the app still leaks every note to every user. This is the honest ceiling of an automated gate.

CHAPTER 38

# Closing the gap

> *The IDOR is catchable -- but only by a check that knows your model.*

## 38.1 What would have caught it

G1 is not uncatchable in principle; it is uncaught by the five generic layers as configured. Each of three additions would have caught it, and each requires knowing something about the application's own authorization model:

1. ["**A targeted acceptance scenario:** 'Given I am alice, when I open bob's note by id, then the response is 403.' One Gherkin scenario turns the IDOR into a BDD catch.", "**An object-level DAST probe:** log in as one user, try to fetch another user's object by id, flag a 200. Generic scanners cannot write this for you because they do not know what 'another user's object' means in your schema.", "**A human code review** that asks, of every route returning an object, 'who is allowed to see this, and where is that checked?'"]

The common thread: the generic gate catches generic flaws, and the flaws specific to your domain need checks specific to your domain. The value of measuring the gate is knowing exactly where that line falls, so the manual effort goes where the tools are blind.

CHAPTER 39

# The quality gate in CI

> *Run all five, fail the build on any.*

## 39.1 One command, one exit code

The gate is what CI runs on every push. It executes the five layers on the current source and exits non-zero if any fails, which blocks the merge.

Listing 26.1 - the gate (gate/quality_gate.py) gate/quality_gate.py

```
def run_all() -> tuple[dict, bool]:
    results = {
        "unit": unit.run(),
        "bdd": bdd.run(),
        "mutation": mutation.mutation_score("app/authz.py", "tests/test_authz.py"),
        "sast": sast.scan("app"),
        "dast": dast.scan(),
    }
    overall = all(r["passed"] for r in results.values())
    return results, overall
```

On the clean app it prints five PASS lines and exits 0. The CI workflow then runs the fault-injection measurement and uploads the reports, so every push both enforces the gate and re-derives the evidence for why all five layers are there.

Listing 26.2 - the CI steps (.github/workflows/ci.yml) .github/workflows/ci.yml

```
- name: Quality gate (unit + BDD + mutation + SAST + DAST)
  run: python -m gate.quality_gate

- name: Fault-injection measurement (which layer catches which defect)
  run: python -m harness.score

- uses: actions/upload-artifact@v4
  with:
    name: reports
    path: reports/*.json
```

CHAPTER 40

# The gate's verdict

> *Five lines and an exit code.*

## 40.1 A readable pass/fail

The gate prints one line per layer with the detail that matters on a failure -- the failing scenarios, the surviving mutants, the findings -- and exits non-zero if any layer failed, which is what blocks a merge.

Listing 26.3 - the per-layer summary line (gate/quality_gate.py) gate/quality_gate.py

```
def _summary_line(layer: str, r: dict) -> str:
    verdict = "PASS" if r["passed"] else "FAIL"
    if layer == "unit":
        detail = f"{r['n_passed']} passed" + (
            "" if r["passed"] else f", {r['n_failed']} failed")
    elif layer == "bdd":
        detail = f"{r['n_scenarios']} scenarios" + (
            "" if r["passed"] else f"; failed: {r['failed']}")
    elif layer == "mutation":
        detail = f"score {r.get('kill_rate', 0):.2f} (threshold {r.get('threshold')})"
        if r.get("survivors"):
            detail += f"; survivors: {r['survivors']}"
    else:
        detail = f"{r['n']} finding(s)"
        if not r["passed"]:
            detail += ": " + ", ".join(
                f.get("id", f.get("probe", "?")) for f in r["findings"])
    return f"  [{verdict}] {layer:9s} {detail}"
```

Listing 26.4 - the gate on the clean app

```
============================================================
 Gauntlet quality gate
============================================================
  [PASS] unit      23 passed
  [PASS] bdd       4 scenarios
  [PASS] mutation  score 1.00 (threshold 0.9)
  [PASS] sast      0 finding(s)
  [PASS] dast      0 finding(s)
============================================================
 RESULT: PASS - all five layers clear
============================================================
```

CHAPTER 41

# The mutation ratchet

> *Why the threshold sits just below the current score.*

## 41.1 A number that only goes up

The mutation gate fails below 0.90. That is not an arbitrary bar -- it is set just under the clean suite's 1.00 so that any real loss of coverage trips it. Teams run mutation testing as a ratchet: the threshold tracks the current score, so a change that weakens the tests fails even if it adds none, and the only way the number moves is up.

Defect T1 is exactly the case the ratchet exists for. The suite stays green, so no ordinary gate reacts; the score falls to 0.75; the ratchet fails the build. Without it, a team can quietly erode its own test coverage for months while every dashboard stays green.

**MUTATION TESTING IS NOT FREE**

It reruns the suite once per mutant, so it is the most expensive layer. Gauntlet keeps it cheap by scoping it to the small, pure authorization module and running in process. Real projects scope it to changed files or run it nightly rather than per-commit.

CHAPTER 42

# Reproducibility

> *The same inputs give the same reports, every time.*

## 42.1 Determinism by construction

Every layer is deterministic. The mutation engine walks the AST in a fixed order. Bandit and the custom rules are pure functions of the source. The DAST probes use a fixed seed and a fixed payload against an in-process client. The harness applies patches by literal replacement and sorts its outputs.

So `reports/catch_matrix.json`, `reports/summary.json` and `reports/agreement.json` are byte-identical on every run -- which is verified by re-running the harness and diffing. A number in this notebook that disagreed with a fresh run would be a bug, not a rounding difference.

CHAPTER 43

# Context: the OWASP classes these map to

> *The toy defects are small instances of the industry's top risks.*

## 43.1 Why these defects and not others

The catalog is not arbitrary. Each vulnerability is a scaled-down instance of a category from the OWASP Top 10, the industry's consensus list of the most common web application risks:

Table 29.1 - defects mapped to OWASP Top 10 (2021)

| Defect | OWASP category |
|---|---|
| S1 SQL injection | A03: Injection |
| S2 stored XSS | A03: Injection (cross-site scripting) |
| S3 hard-coded secret | A05: Security Misconfiguration |
| S4 MD5 hashing | A02: Cryptographic Failures |
| S5 missing admin check | A01: Broken Access Control |
| S6 missing headers | A05: Security Misconfiguration |
| G1 IDOR | A01: Broken Access Control |

Broken access control (A01) is the top risk on that list, and it is also the class with a foot on both sides of this gate's line: S5 is caught by DAST, while G1 escapes entirely. That is not a coincidence -- access control is logic specific to your domain, and generic tools catch the generic half of it.

CHAPTER 44

# Where each layer belongs in a pipeline

> *Cost and feedback speed decide when each runs.*

## 44.1 Not everything runs on every commit

The layers differ by an order of magnitude in cost and in how often they find something, so a real pipeline stages them rather than running all five on every keystroke.

Table 30.1 - staging the layers

| Layer | Cost | Typical stage |
|---|---|---|
| unit | milliseconds | pre-commit and every push |
| bdd | seconds | every push |
| sast | seconds | every push / pull request |
| dast | seconds to minutes | pull request / nightly |
| mutation | minutes (reruns the suite per mutant) | pull request on changed files, or nightly |

Gauntlet runs all five on every push because the app is tiny and every layer finishes in seconds. The structure is what transfers: put the fast, cheap layers early where they give instant feedback, and the slow, thorough ones where a few minutes is acceptable -- but make the slow ones blocking, because the defects they alone catch (a toothless test, a runtime leak) are exactly the ones a team will otherwise ship.

CHAPTER 45

# What this does not establish

> *The honest limits of a ten-defect study on one small app.*

## 45.1 Limits

- ["**One small app.** The absolute catch counts are properties of this catalog and this code, not universal constants. The structure -- each layer owning a class, one class escaping -- is the transferable result, not '2 of 10'.", '**Curated defects.** The defects are chosen to be clean, single-cause instances of their class. Real defects are messier and sometimes span classes; a real SQL injection might also break a feature and be caught functionally too.', "**In-process DAST.** The probes use Flask's test client, not a real HTTP scanner over the network. That makes them reproducible but means they miss anything that only appears under a real server, proxy or browser.", '**Mutation scoped to authz.** The mutation gate judges one module. A weakened test elsewhere would not be caught; mutation coverage is only as broad as where you run it.', '**The gate bounds risk; it does not prove safety.** G1 is the standing reminder. A green five-layer gate means the known generic classes are handled, not that the app is secure.']

None of these weakens the finding; they place it. The project measures what five common layers do and do not catch on a controlled set of defects, with its baseline and its false-positive rate both measured. That is the claim, and no more.

CHAPTER 46

# Exercises

> *Extend the gate and watch the matrix move.*

## 46.1 Problems

1. ["Add a Gherkin scenario that fetches another user's note by id and expects 403. Re-run the harness. Which column does G1 light up in now, and does its catalog prediction need updating?", 'Add a sixth defect: a second weak test that neuters a `can_edit` assertion. Predict which layer catches it before running.', 'Lower the mutation threshold to 0.70. Does T1 still fail the gate? What does that tell you about choosing the bar?', "Write a defect that both breaks a feature and is a vulnerability (e.g. search that is injectable AND returns wrong results). How many layers catch it, and what does that do to the 'each layer owns a class' story?", 'The benign control N1 adds an unused constant. Devise a benign change that you expect to trip a layer falsely, and find out whether it does.']

## 46.2 Answers

1. ['G1 moves into the `bdd` column; its prediction changes from empty to `[bdd]`, and the escaped set becomes empty. The gap was a missing test, not an uncatchable bug.', 'Mutation catches it (the authz suite loses teeth), and only mutation -- the same shape as T1. The acceptance suite stays green.', "At 0.70, T1's score of 0.75 would *pass* (0.75 is above 0.70), so the gate would miss it. The threshold must sit just below the achieved score, or the ratchet has slack a weakened suite can hide in.", "Several layers catch it at once (functional AND sast/dast), which is fine: 'each layer owns a class' is about coverage, not exclusivity. Overlap is defense in depth, not waste.", "Most benign edits trip nothing (that is the point of N1). A change that renames a variable to contain the word 'password' can trip Bandit's B105 heuristic -- a true false positive, and a good lesson in why SAST findings need triage, not blind trust."]

CHAPTER 47

# Recap

> *What to carry out of this notebook.*

## 47.1 The five sentences

1. ['A good green test suite (unit + BDD, 100% mutation score) caught 2 of 10 injected defects -- the functional bugs and nothing else.', 'Mutation testing caught the one defect that was invisible to every other layer: a test that passed while asserting nothing.', 'SAST and DAST are complementary -- SAST sees code-level vulns with no runtime trace, DAST sees runtime vulns with no code pattern, and they confirm each other where a vuln is both.', 'The full five-layer gate caught 9 of 10, with zero false positives on a benign change.', 'One flaw -- an IDOR behind correct, fully tested authorization rules -- escaped all five, because a quality gate bounds risk but cannot prove safety.']

A quality gate is not one tool; it is a portfolio of them, each chosen to see a different kind of wrong -- and a clear-eyed note of the wrong that none of them can see.

CHAPTER 48

# Runbook

> *Run the gate and reproduce every number in this notebook.*

## 48.1 Install and run

Listing R.1 - the whole workflow

```
python -m venv .venv
# Windows: .venv\Scripts\activate   macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt

python -m gate.quality_gate     # the gate on the clean app (5 x PASS, exit 0)
python -m harness.score         # inject 10 defects -> reports/*.json
```

Run any single layer directly: `python -m pytest tests`, `python -m behave`, `python gate/mutation.py`, `python gate/sast.py app`, `python gate/dast.py`.

## 48.2 What you should see

- ['The gate prints five PASS lines and exits 0.', 'The harness prints the defect x layer grid: green suite 2/10, full gate 9/10, G1 escaping all five, no false positives.', 'The three report files are byte-identical on a second run.']

CHAPTER 49

# Appendix - glossary

> *Terms used in this notebook.*

## 49.1 Glossary

Table A.1 - glossary

| Term | Meaning |
|---|---|
| SAST | Static application security testing: analysing source code without running it. Here, Bandit plus custom AST rules. |
| DAST | Dynamic application security testing: attacking the running app and observing responses. Here, probes through Flask's test client. |
| BDD | Behaviour-driven development: acceptance tests written in business language (Gherkin), run by Behave. |
| mutation testing | Introducing deliberate small changes to the code and checking whether the tests fail; measures whether tests have teeth. |
| mutation score | Killed mutants / total mutants; the fraction of deliberate bugs the suite caught. |
| IDOR | Insecure direct object reference: accessing another user's object by guessing or changing its id, because the route does not check ownership. |
| XSS | Cross-site scripting: an attacker's markup is rendered unescaped and runs in another user's browser. |
| quality gate | An automated pass/fail check in CI that blocks a merge when any of its component checks fails. |
| false positive | A gate finding on code that is not actually defective; too many train a team to ignore the gate. |